ParkingHub

Sistema de reserva de vagas de estacionamento
Guia para a apresentação do projeto

Equipe

Paulo Henrique Alves de Almeida

Matheus Avanzo dos Santos

Murilo Roselini

Petterson Augusto Papa de Souza

Raphael Souza Araujo Barbosa

Tecnologias: HTML, CSS e JavaScript puros, sem servidor e sem bibliotecas.

1. O que é o projeto

O ParkingHub é um site onde a pessoa **reserva uma vaga de estacionamento**, como se escolhesse o assento em um cinema. Ela escolhe o período, clica numa vaga da planta, informa seus dados e escolhe a forma de pagamento. Depois pode acompanhar, cancelar ou encerrar a reserva.

É um **protótipo**: não existe servidor nem banco de dados, e nenhuma cobrança real é feita. Tudo é salvo no próprio navegador (`localStorage`), então os dados sobrevivem ao recarregar a página, mas ficam só naquele computador.

Como abrir

Dê um duplo clique em `index.html`. Não precisa instalar nada nem estar online.

Arquivos do projeto

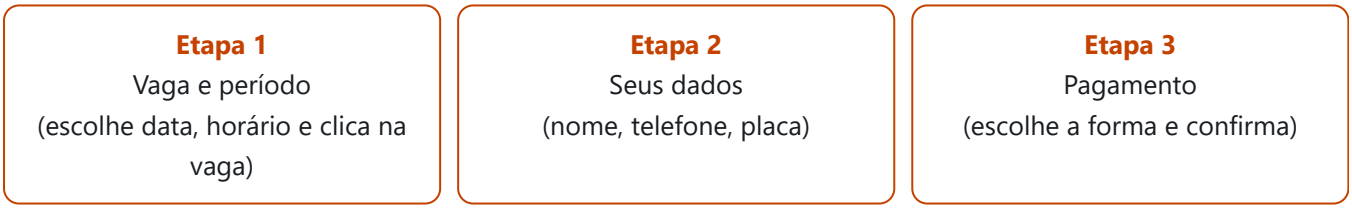
Arquivo	Para que serve	Linhas
`index.html`	**Estrutura**: o que existe na página (formulários, planta, lista, modal).	178
`style.css`	**Aparência**: cores, temas claro/escuro, desenho da planta, layout.	171
`script.js`	**Comportamento**: regras, cálculos, validações, salvamento.	303

> **Ideia central para explicar:** o HTML diz *o quê*, o CSS diz *como parece* e o JavaScript diz *o que acontece*. Os três trabalham juntos, ligados pelos `id` dos elementos.

2. O que o sistema faz

Recurso	Como funciona
Modalidades	**Por horário**: R$ 8,00 por hora (hora iniciada conta como hora inteira). **Diária**: R$ 35,00 por 24 horas.
Planta com 34 vagas	26 de carro (V1 a V26), 6 de moto (M1 a M6) e 2 acessíveis (D1 e D2).
Disponibilidade	Uma vaga fica bloqueada se já existe reserva ativa nela em um horário que se sobrepõe ao escolhido.
Regra da placa	A mesma placa não pode ter duas reservas ativas ao mesmo tempo, nem em vagas diferentes.
Pagamento	Cartão e dinheiro ficam **pendentes** (paga no estacionamento). Google Wallet e Pix são **simulados**. Nunca há cobrança real.
Cancelar	A reserva vira "cancelada" e a vaga volta a ficar livre.
Tema	Claro ou escuro, pelo botão no cabeçalho. Começa de acordo com o sistema do computador.

3. O fluxo do usuário



A reserva **só é criada no último passo**. Se a pessoa voltar antes de confirmar, nada é salvo. As etapas 2 e 3 compartilham a mesma tela: muda apenas qual formulário aparece ao lado do resumo.

Ao confirmar, a pessoa volta para a etapa 1, vê a mensagem de sucesso e a nova reserva aparece em "Minhas reservas".

4. O arquivo index.html

Contém todo o conteúdo da página, organizado em blocos. O JavaScript encontra cada elemento pelo seu `id`.

Bloco	O que é
`<header>`	Logo, nome do sistema e botão de tema.
`<ol id="etapas">`	A barra "1. Vaga e período, 2. Seus dados, 3. Pagamento". O JS marca qual está ativa.
`form#tela-vaga`	Etapa 1: campos de período, a planta (div onde o JS desenha as vagas) e o resumo com o botão "Continuar".
`section#tela-confirmar`	Etapas 2 e 3: resumo à esquerda e, à direita, `form#form-dados` ou `form#form-pagamento` (só um aparece).
`section#minhas`	Lista de reservas e contador de reservas ativas.

Recursos do HTML que usamos

- `type="date"`, `type="time"`, `type="datetime-local"`: o navegador já mostra o seletor de data e hora.
- `hidden`: esconde elementos (por exemplo, o campo de saída some na diária).
- `<svg>` com `<symbol>`: ícones desenhados no próprio HTML, reaproveitados com `<use>`.

5. O arquivo style.css

Temas por variáveis

As cores ficam em variáveis no topo (`--destaque`, `--superficie`, `--texto` etc.). O tema escuro apenas **redefine essas variáveis**, e o resto do CSS não muda.

```
:root { --fundo: #f5f6f7; --destaque: #c44100; ... }
body.escuro { --fundo: #101214; --destaque: #ff7b29; ... }
```

O JavaScript troca o tema apenas ligando e desligando a classe `escuro` no `<body>`.

A planta do estacionamento

A planta é uma caixa com `position: relative`. Dentro dela, três fileiras com `position: absolute` e medidas em porcentagem, o que mantém o desenho proporcional. Cada fileira usa **flexbox** (`display: flex`), que coloca as vagas lado a lado:

- Fileira de cima: pilha de 3 motos + V1 a V6.

- Fileira do meio: 16 vagas em 2 linhas de 8 (D1, V7 a V13, D2, V14 a V20). Usa `flex-wrap` para quebrar de linha.
- Fileira de baixo: pilha de 3 motos + V21 a V26.

Estados da vaga

Visual	Significado	Regra CSS
Branca	Disponível	`.vaga`
Laranja	Selecionada	`.vaga.selecionada`
Cinza	Reservada ou período inválido	`.vaga:disabled`

Layout responsivo

No computador, a planta fica à esquerda e período/resumo à direita (`display: flex`, lado a lado). Abaixo de 1000 px de largura, `flex-direction: column` faz tudo ficar um embaixo do outro. A planta ganha rolagem horizontal para caber no celular.

6. O arquivo script.js

É dividido em 8 blocos numerados, cada um com um comentário no código. É escrito de forma simples: funções comuns, `if` e `for`. Esta é a ordem para explicar.

6.1 Variáveis e salvamento

```
const VALOR_HORA = 8;
const VALOR_DIARIA = 35;
const MS_POR_HORA = 3600000;   // 1 hora em milissegundos
let reservas = [];             // todas as reservas
let vagaEscolhida = null;      // vaga clicada na planta
```

Para mudar os preços, basta alterar as duas tarifas. As datas são trabalhadas em **milissegundos**, o que torna fácil somar e comparar horários. As reservas ficam no `localStorage`: `JSON.stringify` transforma a lista em texto ao salvar e `JSON.parse` transforma de volta ao carregar.

6.2 Desenhando a planta

As 34 vagas não estão escritas no HTML. A função `criarVaga()` cria um botão para cada uma, e os laços `for` de `criarCarros()` e `criarMotos()` repetem o processo. Os botões ficam guardados na lista `vagas`, e assim o JS consegue pintar ou bloquear cada um depois.

6.3 Período e valor: calcularPeriodo()

```
function calcularPeriodo() {
  const inicio = new Date(dia + "T" + entrada).getTime();
  const fim = ehDiaria() ? inicio + 24 * MS_POR_HORA : new Date(dia + "T" + saida).getTime();
  if (isNaN(inicio) || isNaN(fim) || fim <= inicio) return null;
  ...
  valor = Math.ceil((fim - inicio) / MS_POR_HORA) * VALOR_HORA;
}
```

Devolve o início, o fim e o valor, ou `null` se o período for inválido. `Math.ceil` arredonda as horas para cima: 2h30 cobra 3 horas, ou seja, R$ 24,00.

6.4 A regra mais importante: temConflito()

```
function temConflito(campo, valor, periodo) {
  for (const reserva of reservas) {
    if (reserva.status == "ativa" && reserva[campo] == valor) {
      if (periodo.inicio < reserva.fim && periodo.fim > reserva.inicio) return true;
    }
  }
  return false;
}
```

Dois períodos se sobrepõem quando **cada um começa antes de o outro terminar**. A mesma função serve para vaga (`campo = "vaga"`) e para placa (`campo = "placa"`). Reservas coladas (uma termina às

10h e outra começa às 10h) não conflitam.

> **Exemplo:** reserva existente 08h às 10h. Nova das 09h às 11h: começa (09h) antes do fim da antiga (10h) e termina (11h) depois do começo dela (08h), então conflita. Nova das 10h às 12h: 10h não é menor que 10h, então não conflita.

6.5 Etapa 1: atualizar()

Roda a cada mudança no formulário. Ela recalcula o período, percorre as 34 vagas bloqueando as ocupadas, marca a escolhida, atualiza o resumo (entrada, saída, duração, total) e habilita ou desabilita o botão "Continuar". Se o novo período ocupar a vaga já escolhida, a escolha é desfeita com um aviso.

6.6 Navegação entre as etapas: mostrarEtapa()

Uma única função mostra e esconde as telas conforme o número da etapa (1, 2 ou 3) e atualiza a barra de progresso. O botão "Voltar" apenas chama `mostrarEtapa(etapaAtual - 1)`.

6.7 Etapa 2: validação dos dados

Campo	Regra (no código)
Nome	`nome.includes(" ")`: precisa ter nome e sobrenome (um espaço no meio).
Telefone	`isNaN(telefone)` e `telefone.length`: só números, com 10 dígitos (fixo) ou 11 (celular), com DDD.
Placa	`placa.length != 7`: 7 caracteres (cabe o padrão antigo ABC1234 e o Mercosul ABC1D23). Fica em maiúsculas.
Placa em uso	Usa `temConflito("placa", ...)` para impedir duas reservas da mesma placa no mesmo horário.

Cada teste é um `if`. Se algo estiver errado, `mostrarErro()` exibe a mensagem e o `return` interrompe a função, sem avançar para a etapa 3.

6.8 Etapa 3: pagamento e criação da reserva

O objeto `textosPagamento` guarda o texto de cada forma de pagamento. Ao escolher uma, a tela mostra a descrição e, no Pix, o QR Code de demonstração. Ao confirmar, o sistema **confere a disponibilidade de novo** (outra aba poderia ter reservado a vaga) e só então cria o objeto da reserva:

```
{ vaga, nome, placa, pagamento, inicio, fim, valor, status: "ativa" }
```

Esse objeto entra na lista com `reservas.push(reserva)` e é salvo com `salvarReservas()`.

6.9 Minhas reservas e cancelar

- **listarReservas()** monta a lista de trás para frente (a mais nova primeiro). Cada reserva vira uma caixa criada com `createElement`, com o botão "Cancelar reserva" somente se estiver ativa.

- **cancelarReserva()** pede confirmação, muda o status para "cancelada" e a vaga fica livre de novo (o `temConflito` só olha reservas ativas).

6.10 Segurança básica

Os textos digitados (como o nome) são colocados na tela com `textContent`. Isso faz o navegador tratar tudo como **texto puro**: se alguém digitar código no campo nome, ele aparece escrito na tela e não é executado (ataque chamado **XSS**).

7. Limites do protótipo

- Sem servidor e sem banco: os dados existem só no navegador de quem usou e não são compartilhados entre pessoas.
- O pagamento é simulado. Não existe integração com banco, Pix ou Google.
- Se duas pessoas usarem o mesmo computador, as reservas aparecem para ambas.

Como evoluiria: um servidor (por exemplo Node.js) com banco de dados para as reservas compartilhadas, login de usuários, integração com um gateway de pagamento real e mais validações.

8. Perguntas que o professor pode fazer

Pergunta	Resposta curta
Por que não usaram framework ou banco de dados?	O objetivo foi aprender a base da web (HTML, CSS e JS) e manter o projeto simples de abrir e entender. O `localStorage` atende um protótipo.
Como evitam duas reservas na mesma vaga e horário?	A função `temConflito()` compara os períodos. A vaga é bloqueada na planta e a verificação é repetida ao confirmar.
Onde os dados ficam salvos?	No `localStorage` do navegador, como texto JSON.
Como calculam o valor?	Horas arredondadas para cima vezes R$ 8, ou R$ 35 fixos na diária.
O que é um laço (for)?	Uma repetição. Usamos para criar as 34 vagas e para passar por todas as reservas conferindo conflitos.
O que acontece se limpar o navegador?	As reservas são perdidas, pois estão só ali. Num sistema real ficariam no servidor.
Como o tema escuro funciona?	O JS liga e desliga a classe `escuro` no body. O CSS redefine as variáveis de cor quando essa classe existe.
O pagamento é real?	Não. Cartão e dinheiro ficam pendentes; Google Wallet e Pix são só simulação.
Por que não validaram a placa com regras mais rígidas?	Para manter o código simples. Hoje confere o tamanho (7 caracteres). Num projeto maior usaríamos expressões regulares.

9. Sugestão de divisão da apresentação

Pessoa	Parte	O que mostrar
1	Visão geral e `index.html`	Objetivo, tecnologias, estrutura da página e das 3 etapas.
2	`style.css`	Variáveis, tema escuro, desenho da planta com flexbox, layout responsivo.
3	Planta, período e conflito	`criarVaga`, `calcularPeriodo`, `temConflito` e `atualizar`.
4	Etapas 2 e 3	Validações com `if`, pagamento, criação da reserva e `localStorage`.
5	Reservas e demonstração	`listarReservas`, `cancelarReserva`, tema, limites e demonstração ao vivo.

Roteiro da demonstração ao vivo

1. Abrir o site: mostrar a planta com todas as vagas livres e trocar o tema.
2. Escolher a vaga V1 e fazer a reserva completa com Pix.
3. Voltar à etapa 1 com o **mesmo horário**: mostrar a V1 bloqueada.
4. Tentar reservar outra vaga com a **mesma placa**: mostrar o erro.
5. Escolher a diária e mostrar o total fixo de R$ 35,00. Digitar uma saída antes da entrada e mostrar o aviso.
6. Em "Minhas reservas", cancelar uma reserva e mostrar a vaga livre de novo. Recarregar a página para provar que os dados ficaram salvos.